Documentazione Tecnica

Worker di Monitoraggio e Notifiche (start_system_monitor_worker)

Modulo: HealthPACSService **Componente:** System Monitor Worker **Versione:** 1.0.0 Production

1. ARCHITETTURA & SOLUZIONE FLICKERING

L'architettura di monitoraggio e notifiche di **HealthPACS** gestisce lo stato di salute del backend (`HealthPACSService`) e il dispatching degli eventi in tempo reale verso il server Web (`HealthPACSWeb`) tramite un worker asincrono centralizzato.

Eliminazione del Flickering e Isolamento Socket
- **Identificazione del Service (`service_sid`):** Il server Socket.IO registra in modo univoco il SID appartenente al processo backend. Alla disconnessione di un browser client, il server ignora l'evento evitando notifiche broadcast indebite.
- **Risposta Istantanea (`on_connect`):** All'apertura o al refresh di un singolo browser, il server invia lo stato reale in RAM (con tolleranza heartbeat di 45s) unicamente a quel socket (`to=sid`), azzerando i falsi positivi OFFLINE.
- **Centralizzazione nel Worker:** Invia l'heartbeat periodico e consuma in modo transazionale le notifiche attive dal DB.

2. MODELLO ORM NOTIFICHE (`APP_NOTIFICATION_WORKER`)

Di seguito si riporta la struttura del modello ORM `AppNotificationWorkerModel` utilizzato per la tracciabilità transazionale degli eventi sul database:

Campo	Tipo SQL	Vincoli	Ruolo Operativo
`id`	Integer	PK, Autoincrement	Identificativo univoco della notifica.
`notify_key`	String(100)	Unique, Index, Not Null	Chiave identificativa univoca dell'evento.
`notify_type`	String(50)	Not Null	Tipologia notifica (es. `STORAGE_ALERT`).
`is_active`	Boolean	Default: False	Indica se l'evento/allarme è attualmente attivo.
`is_sent`	Boolean	Default: False	Flag di conferma invio via Socket.IO dal worker.

3. SORGENTE ORM MODEL (`APP_NOTIFICATION_WORKER_MODEL.PY`)

```
from sqlalchemy import Column, Integer, String, Boolean, Text, DateTime
from sqlalchemy.sql import func
from library.models import Base

class AppNotificationWorkerModel(Base):
    __tablename__ = "app_notification_worker"

    id = Column(Integer, primary_key=True, autoincrement=True)
    notify_key = Column(String(100), unique=True, nullable=False, index=True)
    notify_type = Column(String(50), nullable=False)
    notify_description = Column(Text, nullable=True)
    permanent = Column(Boolean, default=False, nullable=False)
    is_active = Column(Boolean, default=False, nullable=False)
    is_sent = Column(Boolean, default=False, nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now(), null

    def __repr__(self):
        return f"<AppNotificationWorker(key='{self.notify_key}', type='{self.notify_type}', active={s
```

4. SORGENTE REPOSITORY (`APP_NOTIFICATION_WORKER_REPO.PY`)

```
from typing import List, Optional
from sqlalchemy.orm import Session
from library.models.app_notification_worker_model import AppNotificationWorkerModel

class AppNotificationWorkerRepo:
    def __init__(self, db_session: Session):
        self.session = db_session

    def get_by_key(self, notify_key: str) -> Optional[AppNotificationWorkerModel]:
        """Recupera una notifica tramite la sua chiave univoca."""
        return self.session.query(AppNotificationWorkerModel).filter_by(notify_key=notify_key).first(

    def get_active_notifications(self) -> List[AppNotificationWorkerModel]:
        """Recupera tutte le notifiche attualmente attive nel sistema."""
        return self.session.query(AppNotificationWorkerModel).filter_by(is_active=True).all()

    def get_pending_socket_notifications(self) -> List[AppNotificationWorkerModel]:
        """Recupera le notifiche scattate che devono essere ancora inviate via socket."""
        return self.session.query(AppNotificationWorkerModel).filter_by(
            is_active=True,
            is_sent=False
        ).all()

    def trigger_alert(self, notify_key: str, description: Optional[str] = None) -> bool:
        notif = self.get_by_key(notify_key)
        if notif:
            if not notif.is_active:
                notif.is_active = True
                notif.is_sent = False
                if description:
                    notif.notify_description = description
                self.session.commit()
                return True
        return False

    def resolve_alert(self, notify_key: str) -> bool:
        notif = self.get_by_key(notify_key)
        if notif and notif.is_active:
            notif.is_active = False
            notif.is_sent = False
            self.session.commit()
            return True
        return False

    def mark_as_sent(self, notification_id: int) -> None:
        notif = self.session.query(AppNotificationWorkerModel).get(notification_id)
        if notif:
            notif.is_sent = True
            self.session.commit()
```

5. SORGENTE WORKER (`SYSTEM_MONITOR_WORKER.PY`)

```
import time
import logging
import threading
from typing import Any

from library.repository.app_notification_worker_repo import AppNotificationWorkerRepo
from library.constant_health_pacs import SERVICE_STATUS, SERVICE_STATUS_STATES

logger = logging.getLogger(__name__)

def start_system_monitor_worker(
    service_socket: Any,
    dbmanager: Any,
    shutdown_event: threading.Event,
    interval: int = 10,
    fast_interval: int = 5,
    normal_interval: int = 15,
    **kwargs
) -> threading.Thread:
    """
    Worker centralizzato eseguito in background per Heartbeat e Notifiche DB.
    Accetta 'interval', 'fast_interval', 'normal_interval' o kwargs generici.
    """
    fast_wait = kwargs.get("fast_interval", interval or fast_interval)
    normal_wait = kwargs.get("normal_interval", interval or normal_interval)

    def monitor_loop():
        logger.info(f"[SYSTEM MONITOR] Worker avviato (Intervallo: {fast_wait}s retry / {normal_wait}

        while not shutdown_event.is_set():
            is_connected = False
            if hasattr(service_socket, "sio") and service_socket.sio:
                is_connected = getattr(service_socket.sio, "connected", False)
            elif hasattr(service_socket, "is_connected_event"):
                is_connected = service_socket.is_connected_event.is_set()

            if is_connected:
                # 1. EMISSIONE HEARTBEAT
                service_socket.send_notification({
                    "type": SERVICE_STATUS,
                    "sender": "HealthPACSService",
                    "status": SERVICE_STATUS_STATES["ONLINE"],
                    "message": "HealthPACSService is ALIVE",
                    "timestamp": time.time()
                })

                # 2. DISPATCHER NOTIFICHE DA DB (is_active=True, is_sent=False)
                try:
                    with dbmanager.get_session() as session:
                        repo = AppNotificationWorkerRepo(session)
                        pending_events = repo.get_pending_socket_notifications()

                        for event in pending_events:
                            logger.info(f"[SYSTEM MONITOR] Invio notifiche socket per evento: '{event

                            sent_success = service_socket.send_notification({
                                "type": event.notify_type,
                                "notify_key": event.notify_key,
                                "message": event.notify_description,
                                "permanent": event.permanent,
                                "is_active": True,
                                "timestamp": time.time()
                            })

                            if sent_success:
                                repo.mark_as_sent(event.id)
```

6. INTEGRAZIONE MAIN SERVICE (`HEALTHPACSSERVICE.PY`)

```
# HealthPACSService.py (Estratto di avvio e shutdown controllato)
def main():
    signal.signal(signal.SIGINT, signal_handler)
    signal.signal(signal.SIGTERM, signal_handler)

    try:
        logger.info("Inizializzazione di HealthPACS Service...")

        dbmanager = DatabaseManager()
        if not dbmanager.connect():
            logger.critical("Impossibile connettersi al database.")
            return

        Base.metadata.create_all(bind=dbmanager.engine)
        repos = RepositoryContainer(dbmanager)

        dicom_server = start_dicom_server(dbmanager, repos)
        app = create_fastapi_app(dbmanager, repos)
        api_server = start_api_server(app, host="0.0.0.0", port=8000)

        service_socket = start_service_socket_client(dbmanager)

        # Avvio del worker per Heartbeat e Dispatching Notifiche da DB
        start_system_monitor_worker(
            service_socket=service_socket,
            dbmanager=dbmanager,
            shutdown_event=shutdown_event,
            interval=10
        )

        logger.info("HealthPACS Service avviato con successo.")

        while not shutdown_event.is_set():
            shutdown_event.wait(timeout=1.0)

    except KeyboardInterrupt:
        logger.info("Rilevato CTRL+C. Chiusura in corso...")

    finally:
        logger.info("Inizio procedura di Shutdown...")

        if service_socket:
            logger.info("Invio notifica di spegnimento a HealthPACSWeb...")
            service_socket.send_notification({
                "type": SERVICE_STATUS,
                "sender": "HealthPACSService",
                "status": SERVICE_STATUS_STATES["OFFLINE"],
                "message": "HealthPACSService è in fase di spegnimento controllato."
            })
            service_socket.stop()

        if api_server:
            api_server.should_exit = True
        if dicom_server:
            dicom_server.stop()
        if dbmanager:
            dbmanager.disconnect()

        logger.info("HealthPACS Service arrestato correttamente.")
```